



中国科学技术大学

Lab6 综合实验

刘思宇
2026.5.20



目录

CONTENTS

01

扩展指令集(M)

设计乘除法运算单元

02

扩展指令集(F)

设计浮点运算单元

03

高速缓存

Cache接入流水线

04

分支预测



01

扩展指令集(M)



扩展指令集(M)

- 设计乘除法运算单元,在流水线处理器中支持下面的 RV32M 指令
 - mul, mulh, mulhu
 - div, divu, rem, remu
- 要求:
 - 乘法器能在至多 5 个周期内给出正确结果 (也就是 ban 了循环加法)
 - 乘法器和除法器均不允许使用 IP 核或 *、/、% 运算符



扩展指令集(M) – 1. 乘法与处理器

– 乘法的影响

- 使用加法来实现乘法会消耗大量性能
- 多周期乘法时序较好,但仍未解决性能问题
- 组合电路乘法基本解决性能问题,但可能引入大量延迟

– 除法的影响

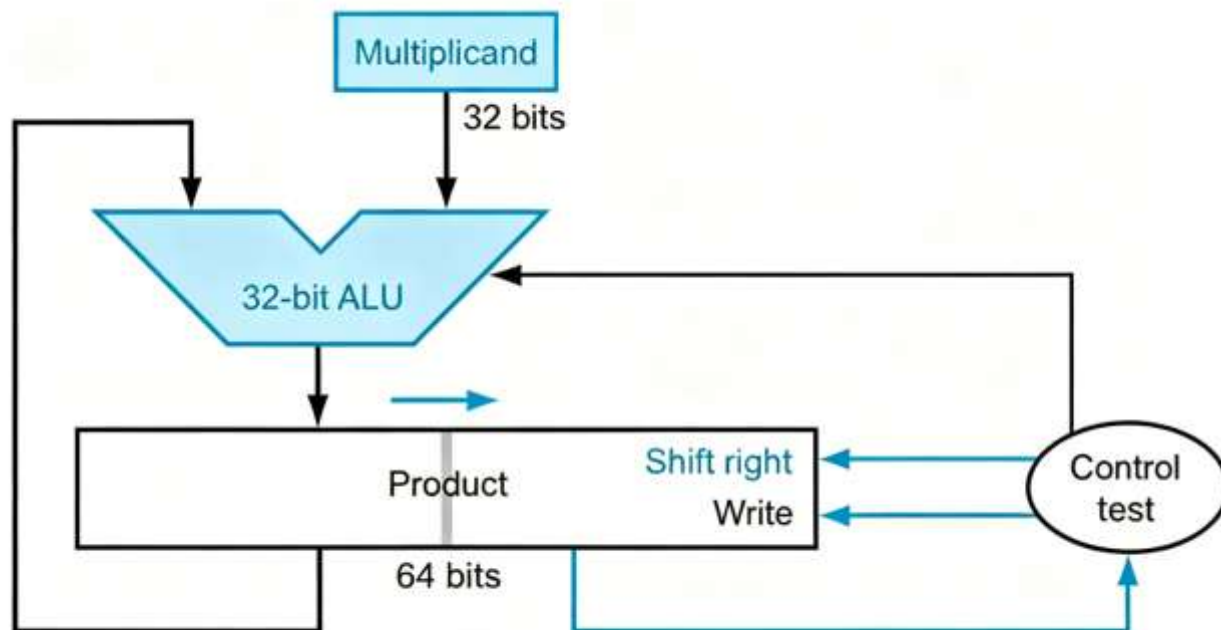
- 除法无法组合化(查表除法消耗太多资源,且表地址较大,延迟相当可观),因此处理除法时不得不停顿
- 除法几乎是无法被替代的运算



扩展指令集(M) – 2.乘法器的设计

– 多周期简单移位乘法器

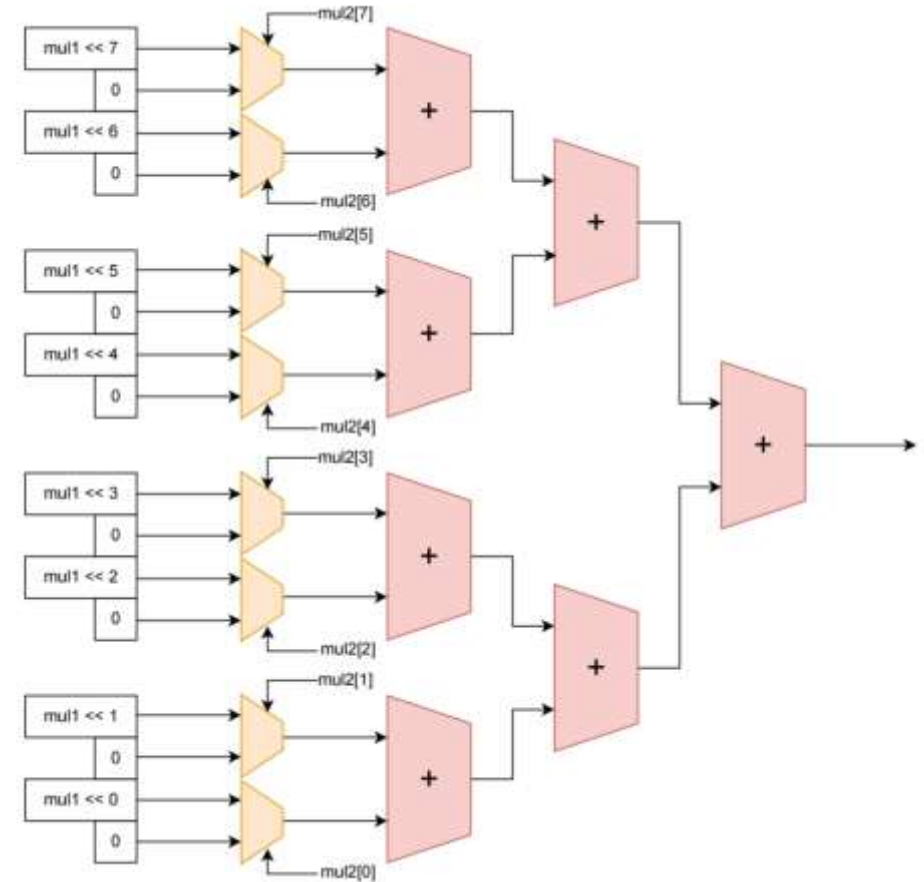
- 借鉴小学乘法思路, 计算32个部分积, 不断将其相加即可
- 时序移位乘法的开销: 32个周期(不合实验要求)
- 硬件的思想: 并行增加带宽, 这其中是否有可以并行的部分?





扩展指令集(M) – 2.乘法器的设计

- 二叉树优化乘法
 - 使用5层二叉树($\log_2(32)=5$), 根据乘数, 将被乘数适当移位, 不断累加至只剩一个结果
 - 为了简单起见每一层二叉树需要使用64位加法器, 加法器可以级内超前进位, 级间串行进位



8bit * 8bit 二叉树乘法示例, 并行计算8个部分积, 根据 $mul2[i]$ 选择输出0还是部分积, 两两相加得到最终结果



扩展指令集(M) – 2. 乘法器的设计

– 超前进位真的超前吗？

- 虽然不再依赖前一位的进位,但越高位进位,逻辑运算次数越多
- 可以看到公式依然存在**依赖关系**,即便可以用MOS管实现多输入与门,但因为电压的因素,一般4个输入的与门已经很危险了
- 64位加法器,假设每4位超前进位,那么相当于16位串行进位,5层超前进位的延迟依然非常巨大(远超ALU的延迟)

$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + P_1 \cdot C_1$$

$$= G_1 + P_1 \cdot (G_0 + P_0 \cdot C_0)$$

$$= G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$$

$$C_{i+1} = G_i + P_i \cdot C_i$$

$$C_3 = G_2 + P_2 \cdot C_2$$

$$= G_2 + P_2 \cdot (G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0)$$

$$= G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

$$C_4 = G_3 + P_3 \cdot C_3$$

$$= G_3 + P_3 \cdot (G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0)$$

$$= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$$



扩展指令集(M) – 2. 乘法器的设计

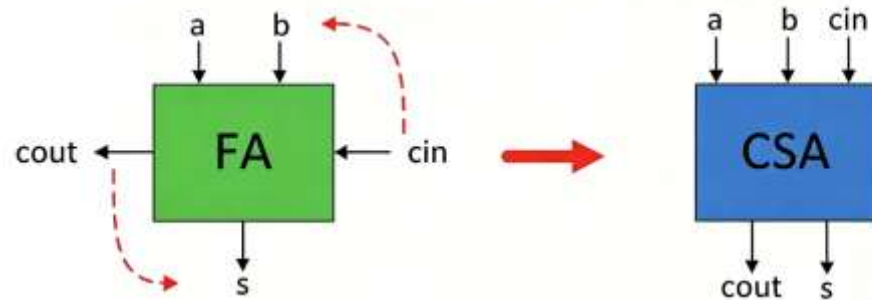
– 保留进位加法器(CSA)

– 逻辑表达式:

$$S = A \oplus B \oplus C$$

$$Cout = AB + BC + AC$$

- 把三个数的和转化为两个数的和，每过一层保留进位加法器，部分和数量减少2/3, 因此也只需要 $\log_{1.5} n$ 层保留进位加法器就可以完成计算
- 最后一层两个数相加应用一个真正的加法器计算





扩展指令集(M) – 2.乘法器的设计

– 2位Booth编码

- 2 位 Booth 编码可以通过有效的编码来缩减乘法中部分积的个数, 由 32 个部分积转化为16 个部分积

- 原理: 针对**有符号乘法** $X * Y$, Y 可以用如下公式变换

$$\begin{aligned} & \boxed{-y_{31}} \times 2^{31} + y_{30} \times 2^{30} + y_{29} \times 2^{29} + \dots + y_1 \times 2^1 + y_0 \times 2^0 \\ &= (y_{29} + y_{30} - 2 \times y_{31}) \times 2^{30} + (y_{27} + y_{28} - 2 \times y_{29}) \times 2^{28} + \dots \\ &+ (y_1 + y_2 - 2 \times y_3) \times 2^2 + (y_{-1} + y_0 - 2 \times y_1) \times 2^0 \end{aligned}$$

- 有了这样的新表示, 就可以使用编码器对 2 的幂之前的系数 X' 进行编码(下一页)



扩展指令集(M) – 2.乘法器的设计

- 2位Booth编码
 - 原系数取值范围： $\{0, 1\}$
 - 2位Booth编码后的系数取值范围： $\{-2, -1, 0, 1, 2\}$

Y_{i+1}	Y_i	Y_{i-1}	输出
0	0	0	0
0	0	1	+X
0	1	0	+X
0	1	1	+2X
1	0	0	-2X
1	0	1	-X
1	1	0	-X
1	1	1	0



扩展指令集(M) – 2.乘法器的设计

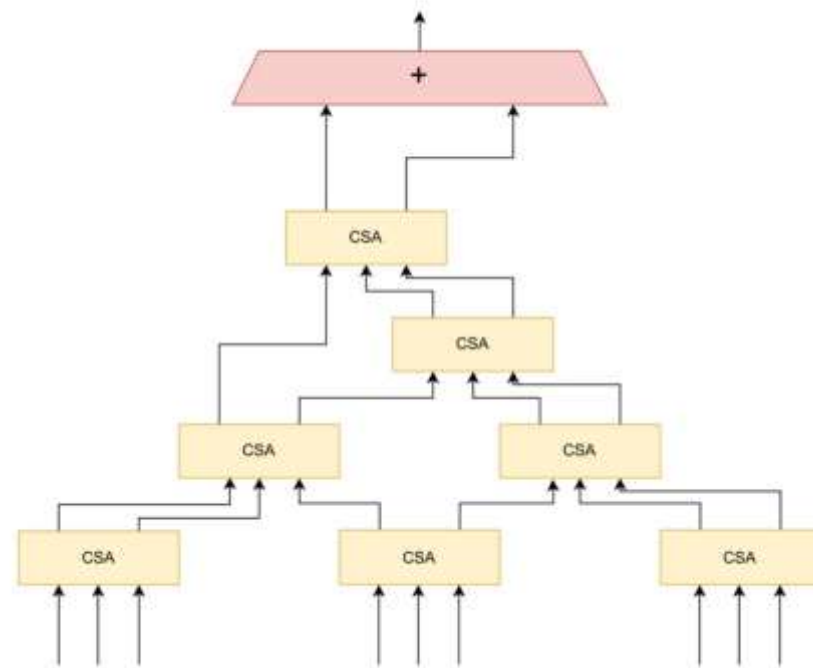
- 2位Booth编码过程
 - 设需要计算 $X*Y$
 - 编码器的输入：Y的相邻3位和X
 - 编码器的输出：处理后的 X'
 - X' 再乘以对应的 2^{2i} 就是一个部分积
 - 32位乘法器需要16个编码器, 编码之后将32个部分积转化为16个部分积



扩展指令集(M) – 2. 乘法器的设计

– 简单华莱士树乘法器

- 树形保留进位加法器的硬件结构：通过不停地使用保留进位加法器,来使得每一层部分积减少1/3,最终缩减为2个数地加法,再通过一个全加器获得最后的结果(注意：对于每个CSA的Cout,应该左移一位再送到下一级CSA)
- 在进入华莱士树之前,我们可以使用2位Booth编码来大幅度缩减部分积的个数
- 32位无符号乘法:为了保证最高位进位不丢失,构造33位华莱士树即可



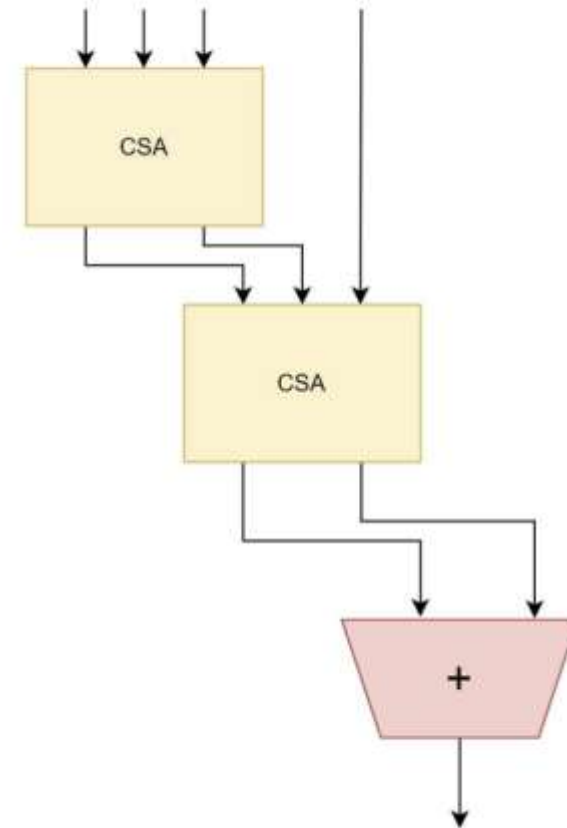


扩展指令集(M) – 2.乘法器的设计

– 简单华莱士树乘法器

– 例：1101*1011 = 10001111

00001101	1101 << 0	
00011010	1101 << 1	
00000000	1101 << 2	
00010111	S1	
00001000	C1	
	00010111 S1	
	00010000 C1 << 1	
	01101000 1101 << 3	
	01101111 S2	
	00010000 C2	
	01101111 S2	
	00100000 C2 << 1	
	10001111	Result





扩展指令集(M) – 3. 除法器的设计

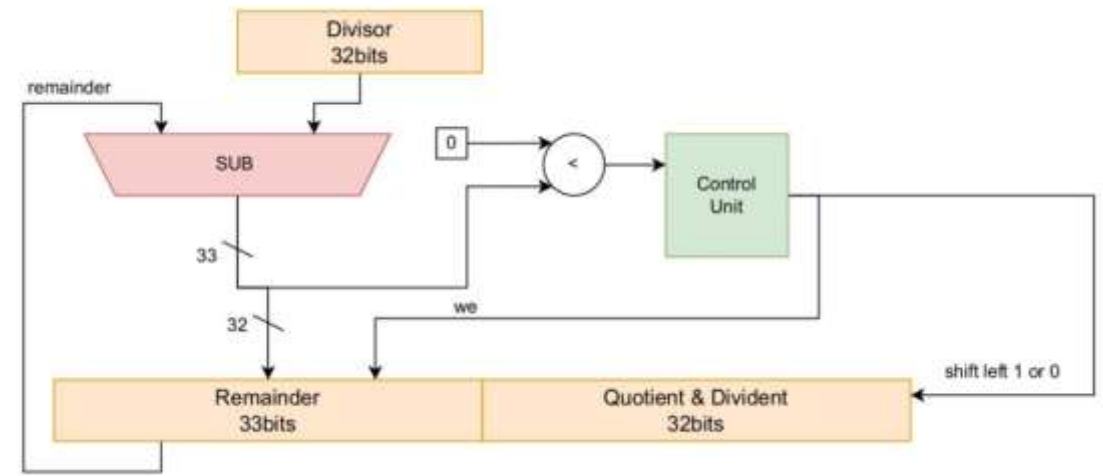
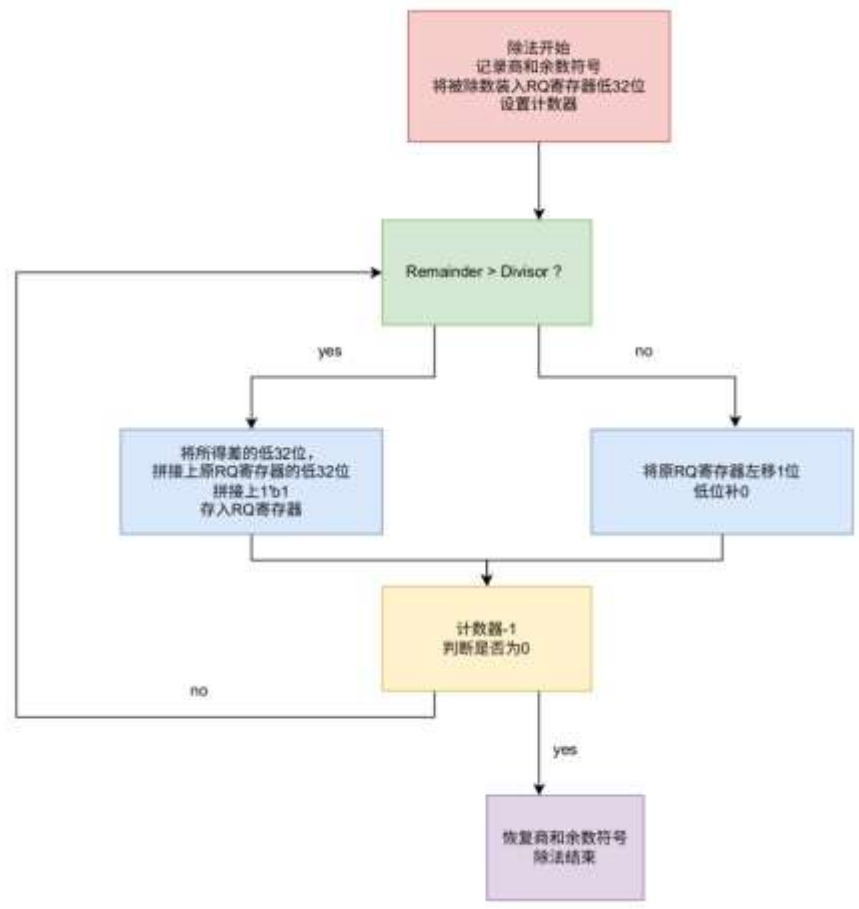
– Radix-2移位除法器

- 使用小学除法竖式计算的思路，使用**多周期**对除法进行计算
- 由于需要多周期，除法器必须由**状态机**控制，并和流水线功能单元进行握手
- 符号处理：在开始除法前，将符号记录下来，并把所有源操作数转换为其绝对值，除法完成后再变换操作数的正负性



扩展指令集(M) – 3. 除法器的设计

– 除法器的逻辑控制

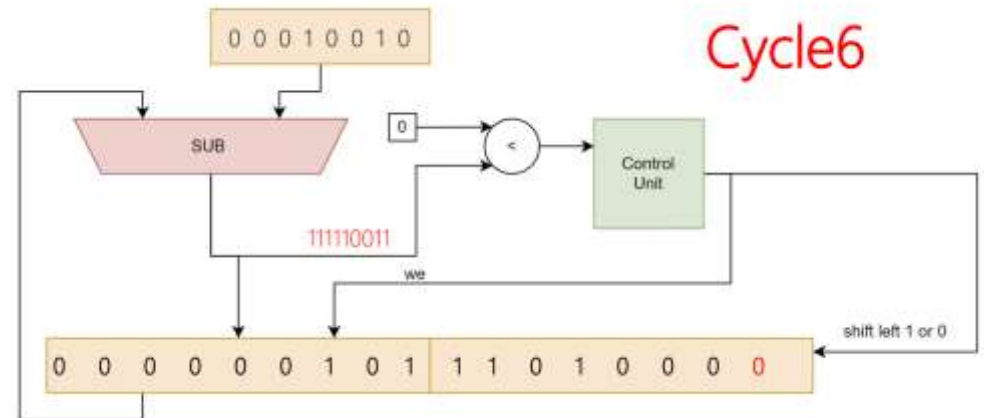
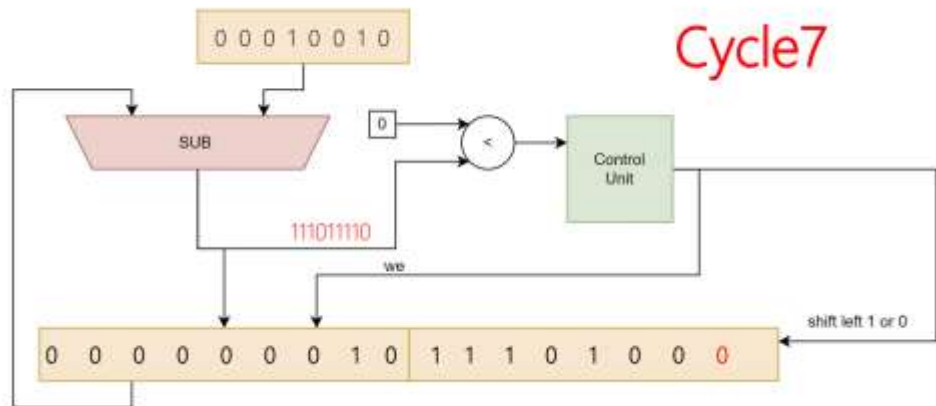
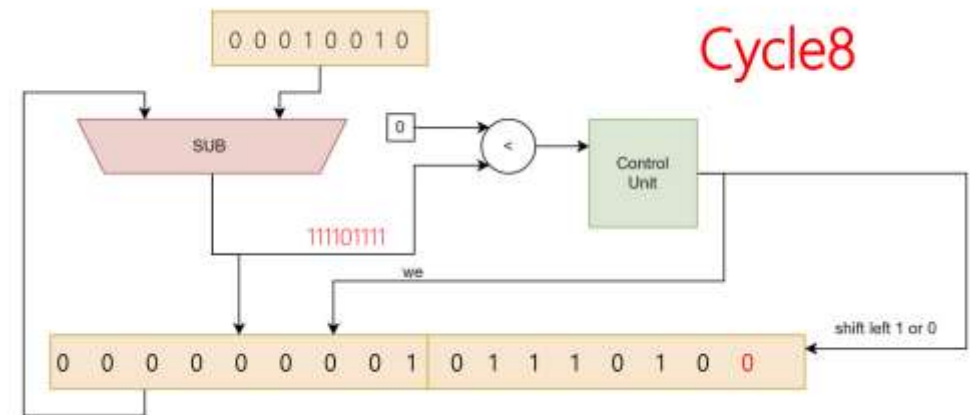
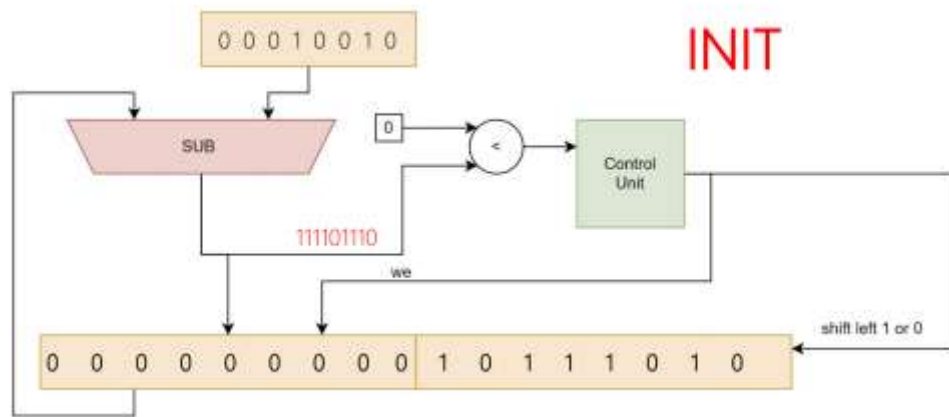


- 初始时要把被除数绝对值放到RQ寄存器低32位
- 不断试余数和除数差（一直假设在商的位置上写1）如果差小于0，那么这一位商为0，需要左移RQ寄存器，来看下一位；如果差大于0，那么把差的低32位放到RQ寄存器的高32位，并将剩余所有低位左移，低位补1
- 除法结束后，注意余数此时在最高32位中（而不是在RQ[63:32]中），恢复余数和除数的符号，除法完成
- 注意：中间部分并不用状态机实现，中间部分完全可以直接进行逻辑判断，除法器的状态机主要是和流水线握手并发送除法开始信号
- 时间开销：64周期



扩展指令集(M) – 3. 除法器的设计

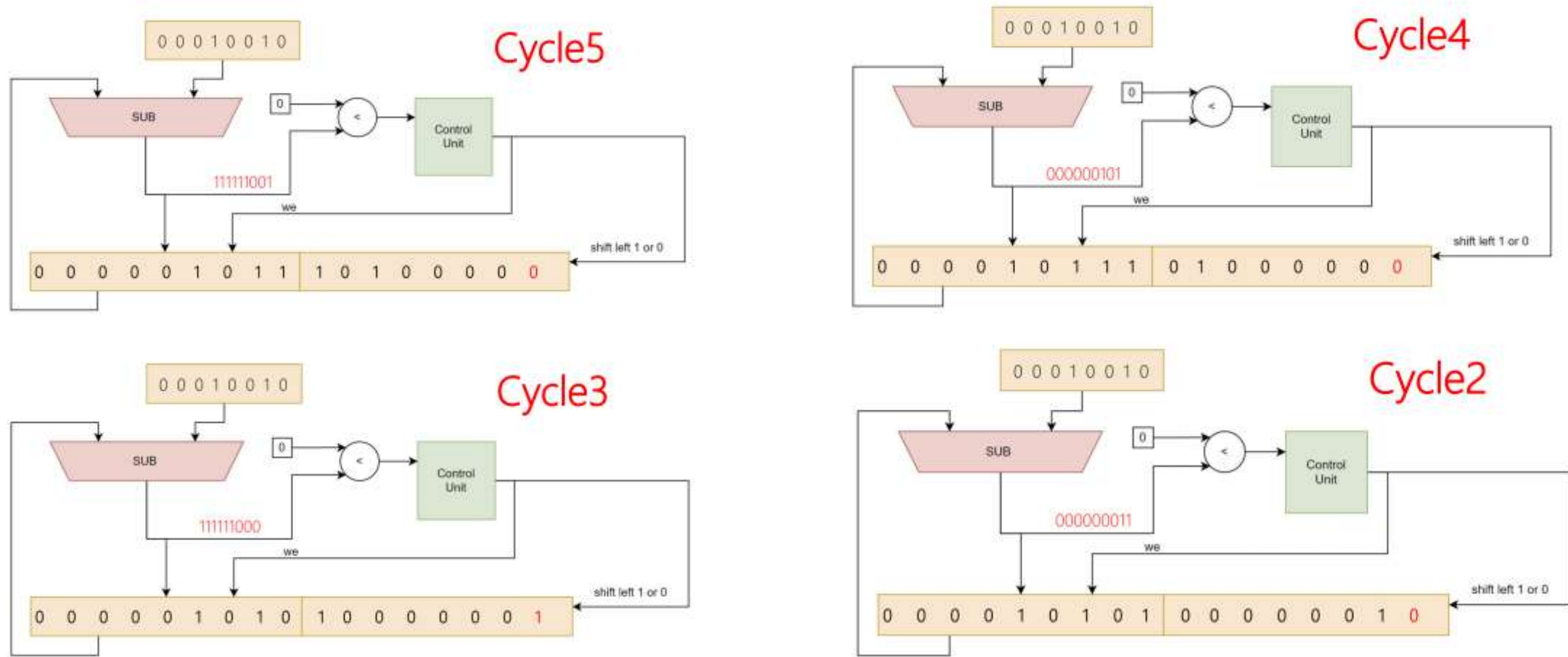
– Radix-2移位除法器过程示例





扩展指令集(M) – 3. 除法器的设计

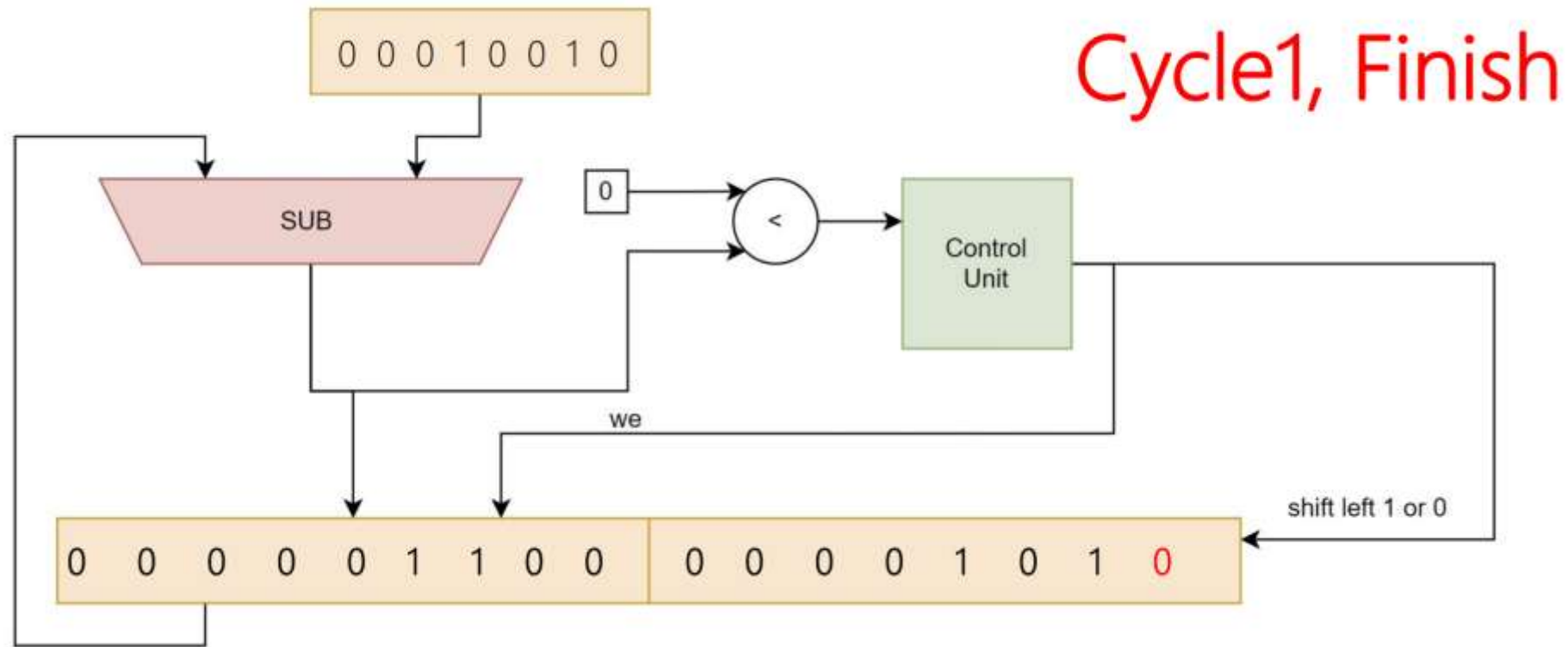
– Radix-2移位除法器过程示例





扩展指令集(M) – 3. 除法器的设计

– Radix-2移位除法器过程示例





扩展指令集(M) – 4. 除法器的优化

– 除法器的优化

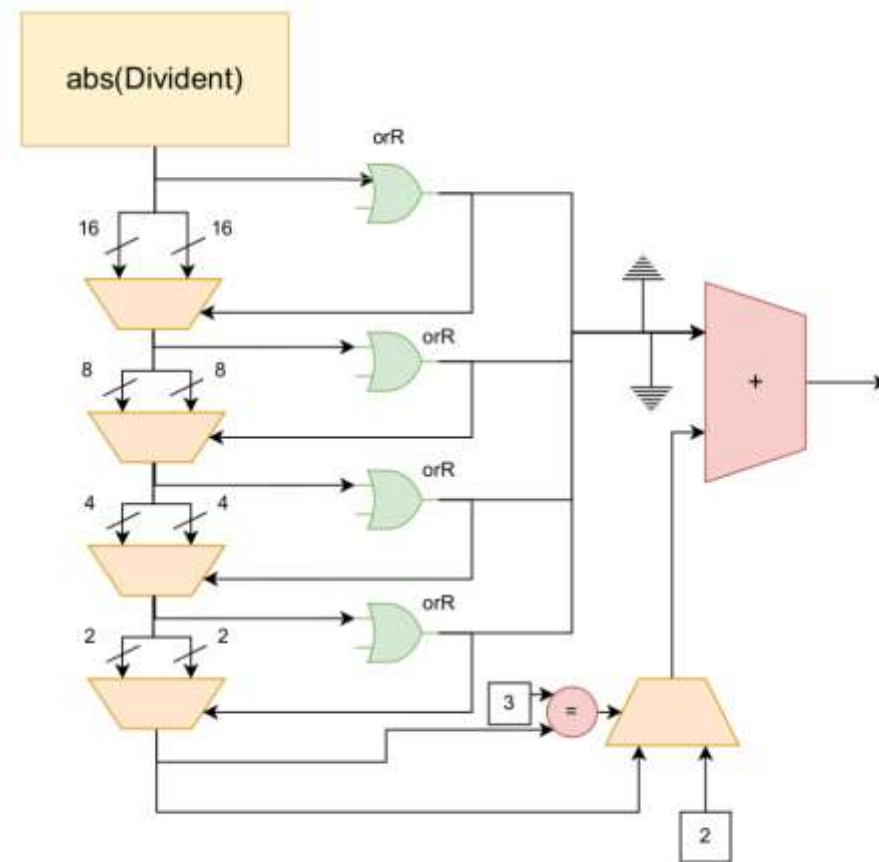
- 在实际应用的大部分除法中，除数和被除数都不会很大，提前启动优化就是针对被除数绝对值较小的情况进行优化
- 重要思想：加速经常性过程
- 提前启动：找出被除数有多少**前导 0**，忽略它们直接从第一个非零数开始试商
- 主要问题：找出被除数高位一共有多少0——求以2为底的对数取上整(对数器)



扩展指令集(M) – 4. 除法器的优化

– 对数器的设计

- 使用二分查找计算出一个数的对数取上整
- 先看这个数最高 16 位是否有 1（可以通过缩位或运算实现），这个结果就是对数中第 4 位的值，如果没有 1，那么取出低 16 位；如果有 1，取出高 16 位
- 再看取出的 16 位数中最高 8 位是否有 1，这个结果就是对数中第 3 位的值，如果没有 1，那么取出低 8 位；如果有 1，取出高 8 位
- 以此类推，直到还剩两位数
- 对于最后两位数，如果它的值是 3，我们需要把结果加上 2，否则，需要加上这个两位数，如此就得到了对数上整的值





扩展指令集(M) – 4. 除法器的优化

- 除法器的提前启动优化
 - 将计数器置为被除数对数上整
 - 将被除数右对齐到RQ寄存器的 $32 - \log_2 \text{Divident}$ 位置



02

扩展指令集(F)



扩展指令集(F)

- 设计浮点运算单元,在流水线处理器中支持下面的 RV32F 指令
 - `fadd.s`, `fsub.s`
 - `fcvt.s.w`, `fcvt.w.s`
- 你可以根据需要设定指令的舍入模式（视作常量,而无需 CPU 保存,意即只需固定实现一种舍入模式,无需考虑多种）
- 同样地, 浮点运算的实现均不允许使用 IP 核



扩展指令集(F)

- 32位单精度浮点数
 - 1bit 符号(S) | 8bit 指数(E) | 23bit 尾数(M), 隐含整数位为1
 - $\text{Value} = (-1)^S \times (1.M)_2 \times 2^{E-127}$
 - 特殊值:
 - ± 0 : 符号位0或1, 其他位全0
 - 无穷大($-\infty, +\infty$): 符号位0或1, 指数E全1, 尾数M全0
 - NaN(非数): 指数E全1, 尾数M不全为0
 - 非规格化数(非常接近0的小数): 隐含整数位为0, 指数E全0, 尾数M不全为0
 - 针对特殊值, 浮点运算指令需要适当处理
- 设计实现浮点处理单元FPU, 其中包含浮点算数单元, 整数浮点转换单元, 并接入流水线CPU



03

高速缓存



高速缓存

- 将 Lab5 设计的数据高速缓存接入流水线
 - 目前 PDU 内置的存储器是没有读写延迟的，为了与 Lab5 的 Cache 适配，你需要修改部分框架代码



04

分支预测



分支预测

- 请结合相关的资料，实现下面的内容：
 - 基于局部历史的分支预测
 - 设计 BHT 与 PHT 的结构，并接入流水线 CPU，正确修改预测失败后流水线与分支预测器的相关内容
 - 基于全局历史的分支预测以及竞争处理
 - 设计全局预测器的结构，并设计算法处理全局预测与局部预测结果不一致时的竞争关系，以及对于全局预测器的实时修正



分支预测

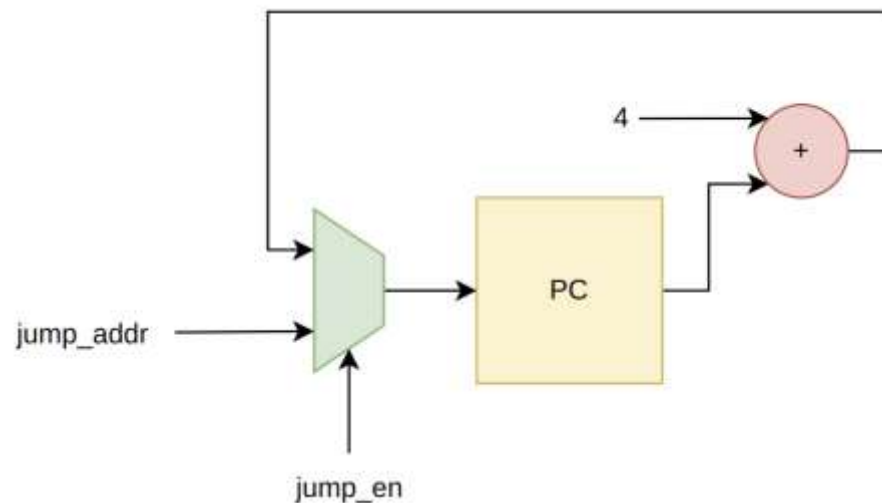
- 为什么需要分支预测

- 为了处理指令之间的相关问题，在 EX 阶段跳转是主流的实现方式
- 分支指令在指令流中虽然不那么常见，但往往是在关键的地方出现
- EX 阶段一个跳转就需要冲刷掉两条指令——着实可惜
 - `for(int i = 0; i < 10000000; i++)`
- 对于一个只会因为控制冒险而冲刷两条指令的流水线，如果一个代码执行流程中有15%的指令是确实跳转的分支指令，那么CPI就从1增加到 $0.85 + 0.15 \times 3 = 1.3$
- 高性能处理器往往会使用超深流水线来减少延迟，一旦分支预测错误，代价可能是非常恐怖的



分支预测 - 1. 设计基础

- 一个简单的分支预测器
 - 实际上是永远预测不跳转的分支预测器
 - 永远认为分支指令不会跳，直接按 $PC+4$ 继续取指
 - 只有当 EX 阶段算出 $jump_en=1$ ，才会把 PC 改成 $jump_addr$ ，冲刷掉前面取的指令

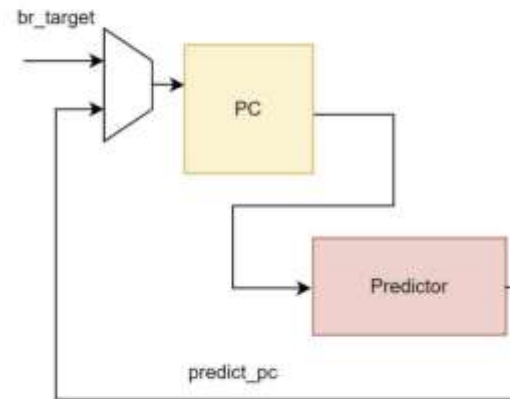




分支预测 – 1. 设计基础

– 分支预测的时机

- IF阶段PC+4, 在ID阶段预测
 - 可以分辨出是跳转指令并计算出跳转地址, 但是下一条指令仍然进入了流水线, 需要跳转的时候仍要flush
- 在EX阶段预测
 - 还不如不预测
- IF阶段预测
 - 直接组合读出来并进行预测
 - 需要能够纠正
 - 当分支预测在 IF 阶段完成后, “跳转信号” 本身不再是触发流水线冲刷的信号了, 冲刷只在预测错误时才发生, 和是否跳转无关

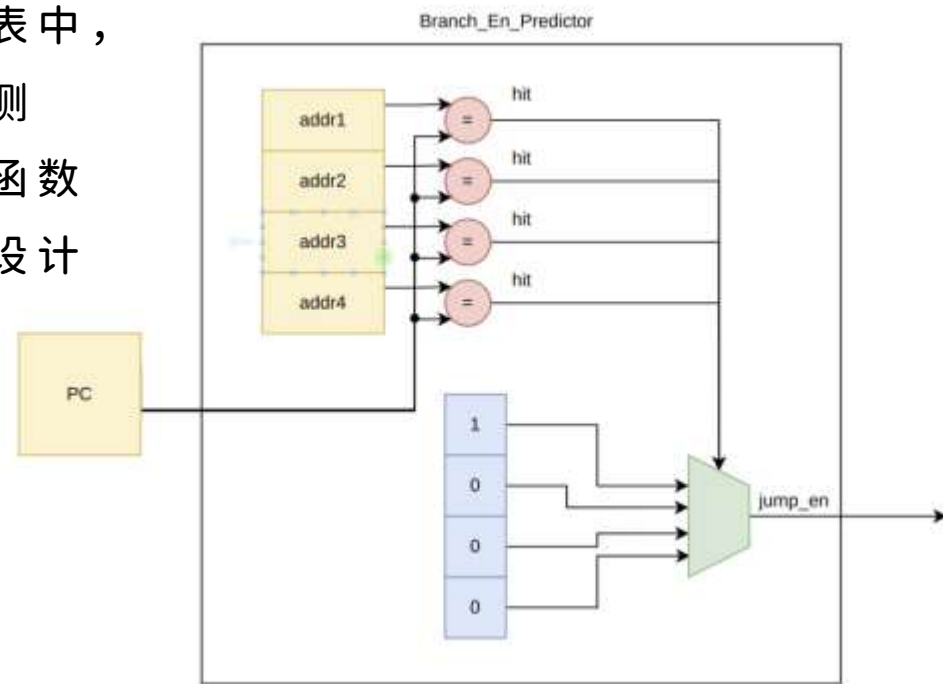


IF-ID



分支预测 - 2. 预测分支方向

- 局部预测 - 基于最后一次结果进行预测
 - 使用一个**1位历史表**，记录最后一次是否跳转
 - 使用高速缓存的思想：查询对应地址的数据是否在表中，如果在，那么根据其对应的历史表的值直接进行预测
 - 由于分支指令的分布是稀疏的，因此需要使用哈希函数对存储器的地址进行映射，或者使用较高相连度的设计（全相连）进行存储

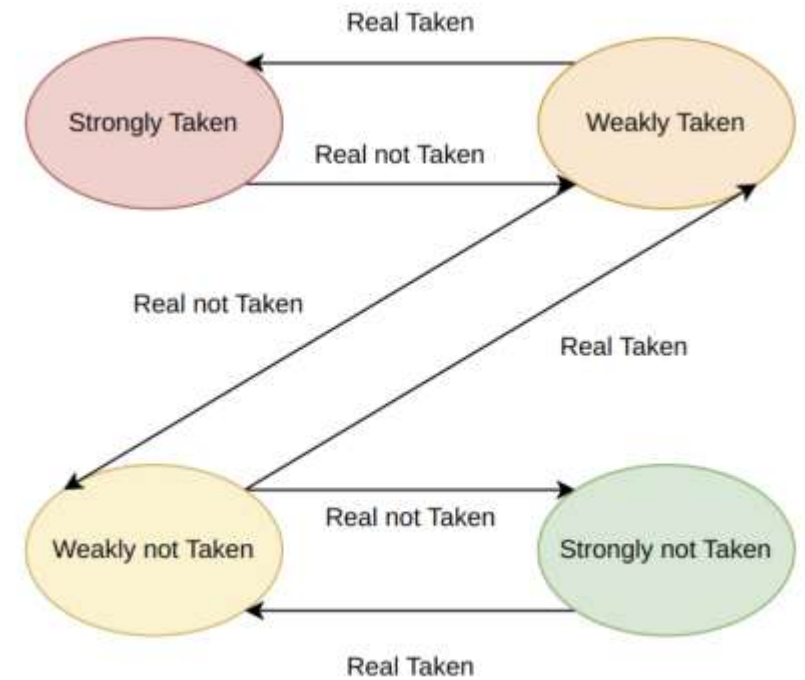


全相联局部预测参考设计



分支预测 – 2. 预测分支方向

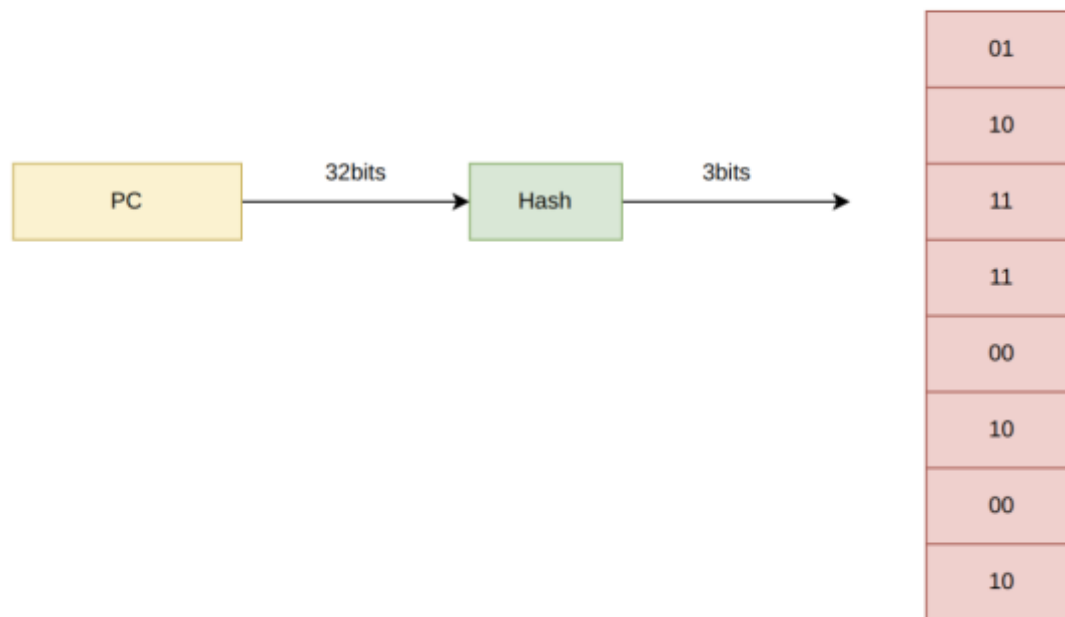
- 局部预测 – 基于两位饱和计数器的预测
 - 在两个 Taken 状态，每次预测其跳转，否则预测其不跳转
 - 根据 EXE 阶段的真实结果来更新状态
 - 针对多重循环非常有效：即便内层循环 执行末尾的那一次预测失败（状态变为 10），下一次进入内循环后依然会预测其跳转





分支预测 – 2. 预测分支方向

- 局部预测 – 基于两位饱和计数器的预测
 - 一种贪心算法的实现
 - 为每一条跳转指令都维护一个2bit计数器，从而得到相对准确的预测，但如此开销巨大
 - 可以使用哈希函数对PC进行哈希从而把32位映射为k位，这样只需维护较少的计数器，用哈希地址进行索引，代价是可能出现哈希冲突导致预测准确度降低





分支预测 – 2. 预测分支方向

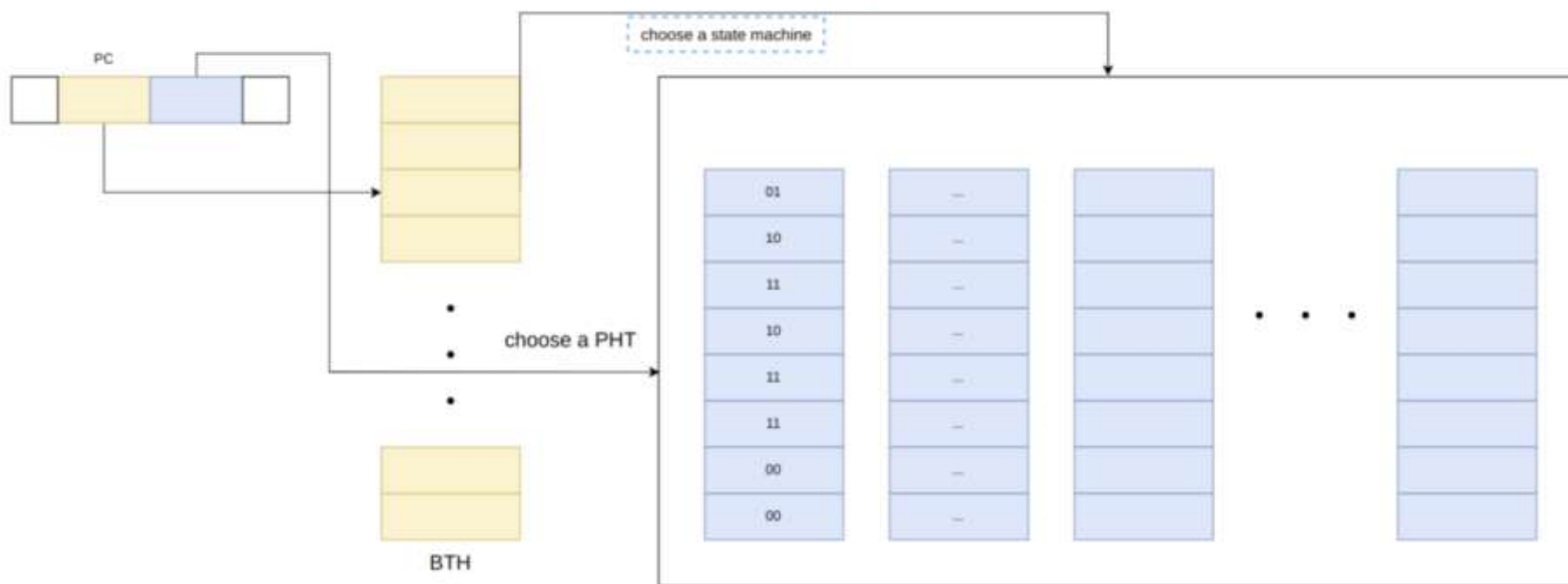
– 局部预测 – 基于局部历史的预测

- 假设一种情况：初始时是 01 状态，之后的跳转情况是 TNTNTNTNT，那么基于 2bit 的分支预测准确率接近于 0（这你会预测吗？）
- 历史信息记录表BHT：记录一条跳转指令过去几次的跳转行为
- 用一条指令的历史信息来索引一个状态是更加合理的
 - 使用指令地址来索引历史信息 BHT
 - 使用历史信息来索引跳转状态 PHT
 - 如果能为每一条指令都维护一个 BHT 和 PHT，那可以相当准确的进行预测，可惜...



分支预测 – 2. 预测分支方向

- 局部预测 – 基于局部历史的预测
 - 可以维护一个 BHT 表和若干 PHT，用不同的地址部分分别进行索引
 - 卡跳转范围：普通跳转指令的跳转范围有限，我们可以维护较多的BHT表项，从而可以倾向于认为：对应到同一个BHT表项的跳转指令基本不冲突
 - 过程：PC 地址 → 选 BHT 条目 → 选 PHT 表 → 用历史读计数器 → 预测&更新





分支预测 – 2. 预测分支方向

– J_{al} 和 J_{alr} 的跳转预测

- 和 branch 一起预测：简化了数据通路设计，但会对分支指令预测结果产生影响（一旦和 br 冲突那么会让准确率急剧下降）
- 单独预测：只要记录下了 J_{al} 的地址，就完全可以相信这个 PC 处会产生跳转（但也要小心自修改程序，指的是运行过程中可能会修改自身指令的程序）



分支预测 – 2. 预测分支方向

– 全局预测

- 上述方法都是基于局部历史进行预测,但遇到下面的代码也会束手无策:

```
if(a==0) temp = 1;
```

```
if(b==0) temp = 2;
```

```
if(a!=b) temp = 3;
```

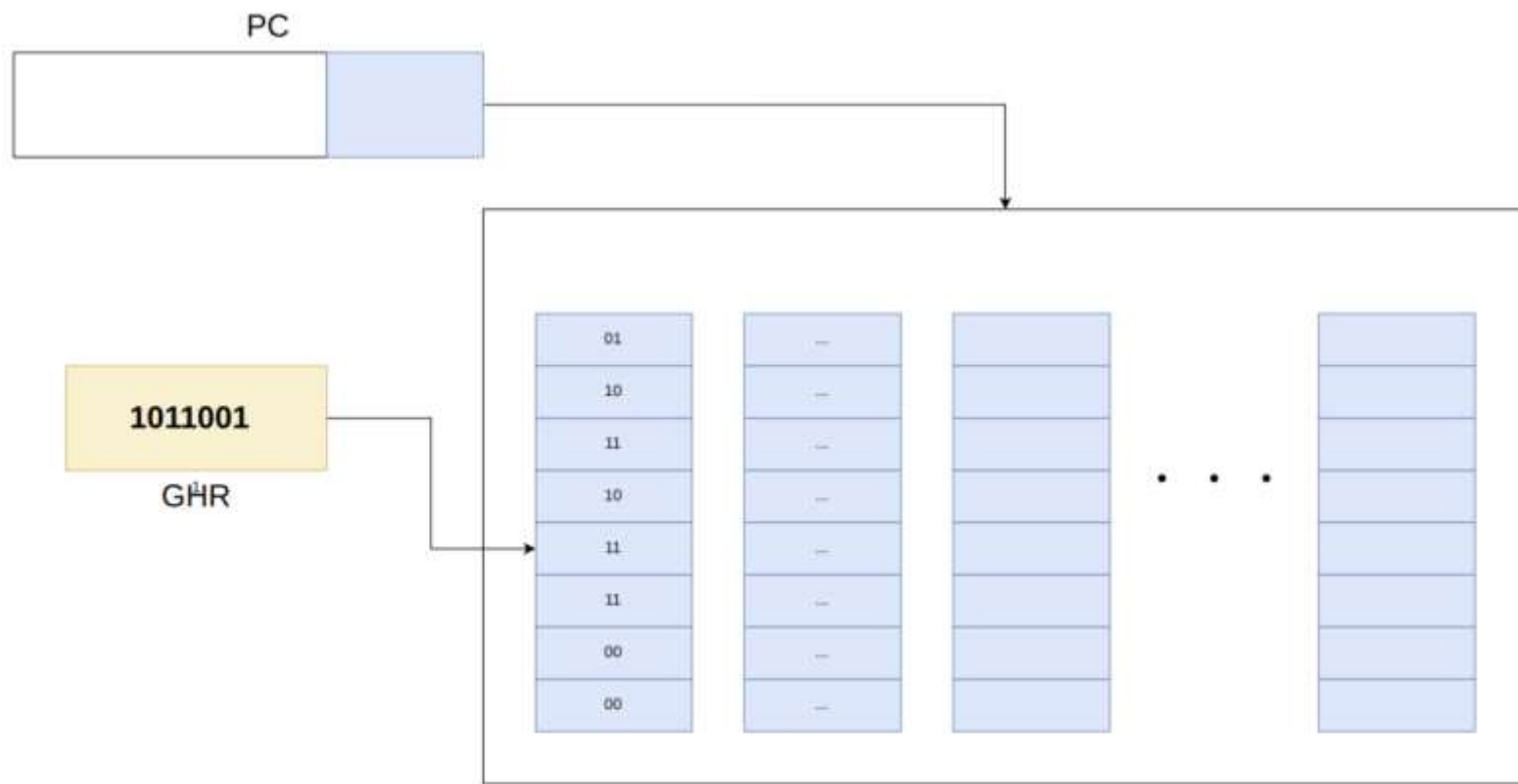
- 可以看出,当前两个分支都不跳转时,第三个分支一定会跳转,但这并不能从第三条分支指令的历史中体现出来,需要全局预测进行补救



分支预测 - 2. 预测分支方向

- 全局预测的实现

- 使用类似于局部预测的方法，使用全局历史寄存器（GHR）来保存整个程序最近几条分支指令的跳转情况，并使用 PHT 进行预测
- 过程：PC选PHT表 -> GHR索引PHT里的行 -> 预测&更新

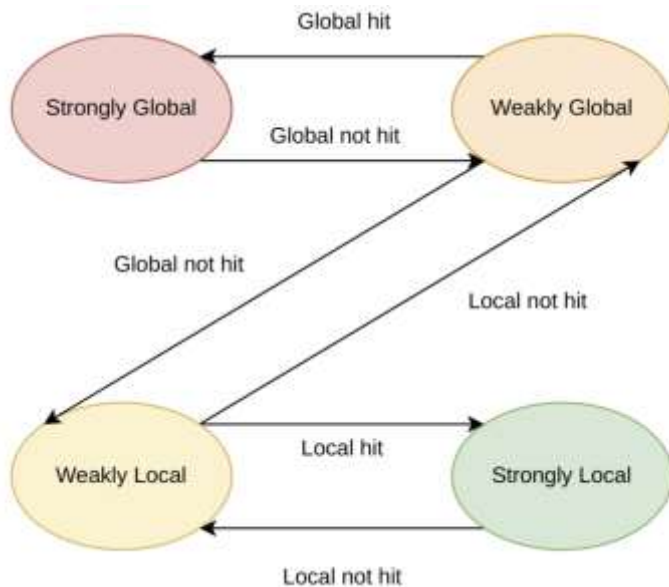




分支预测 – 2. 预测分支方向

– 竞争预测

- 全局预测和局部预测很容易给出不一致的预测结果，这个时候应该听谁的？
- 全局预测和局部预测也会给出一致的结果，那下一次应该听谁的？
- 竞争策略是多种多样的，这里介绍一种简单的竞争策略：
- 所有的分支预测策略必须要理论分析、实验验证，并指出你的优化针对哪些情况做了优化，这些特定情况是否有必要优化或是否为常见情况



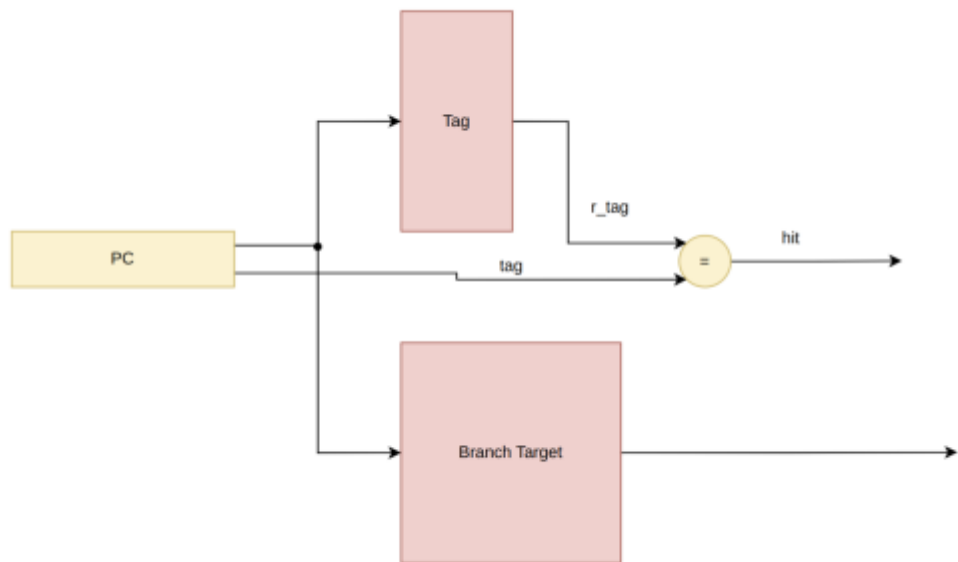


分支预测 – 3. 预测分支地址

– 如何在预测跳转时给出正确地址

– 直接跳转地址预测

- 使用类似于 Cache 的方法保存跳转地址，这个表被称作 BTB (Branch Target Buffer)
- BTB 使用 PC 的低位作为索引，高位作为 tag 判断是否命中
- 若命中，则看方向预测器是否预测跳转，若是，则给出对应地址，否则给出 PC+4
- 由于分支指令的分布是稀疏的，且容易造成冲突，故可以选用合适的哈希函数对 PC 值进行哈希，同时也可以选择多路组相连的方法减少冲突





分支预测 – 3. 预测分支地址

– 跳转（间接跳转）指令地址预测

- Jalr指令的跳转预测难度较大。一般编译器会将 Jalr 用作return指令，而return回去的位置往往难以预料：

```
QuickSort(a,0,i-1);
```

```
QuickSort(a,i+1,n-1);
```

- 一个简单的方法是不预测：不要使用直接跳转的方法，因为 Jalr 容易干扰正常的分支预测
- 相信编译器：对于 99%的情况，哪怕是巨大如操作系统，Jal 的跳转范围也足够，因此编译器只会用 Jal 作函数调用，用 Jalr 作函数返回。因此我们可以用一个栈结构来保存 Jal 指令的下一条指令地址，在执行 Jalr 时弹出地址用作预测地址。同时在 BTB 中保存指令类型以方便预测



分支预测 – 4. 分支预测的纠正

- 之前的流水线中，只要是跳转就会冲刷流水线，但现在不同了，在EX阶段，分支单元还需要知道几件事情：
 - 分支预测给出的预测跳转地址是什么
 - 分支预测器是否是对跳转地址进行预测
- 考虑自修改指令，如果一条branch指令被修改为算数指令，那么这样的跳转是错误的，ID单元就可以纠正过来，并把PC+4进行复位
- **EX 单元必须能纠正所有错误**，只需要比较 NPC 和自己计算的地址是否相同。如果你的分支预测器比较好，可以在不跳转时给出 PC+4，那么就可以对每一个指令（哪怕不是分支指令）进行比对，这样更容易防止分支预测器的错误预测



中国科学技术大学

Thanks !